

Deep Learning Exam vs. Homework Analysis

Prompt:

exam_5 - PDF

exam_7 - PDF

exam_8 - PDF

exam_9 - PDF

hw1 - PDF

I gave it a few samples of the exam and the homework.

I didn't want to overwhelm the LLM, so I used several prompts before the "write the problems" prompt to set it up for success.

You're a teaching assistant for a deep learning course who specializes in exam writing. You will write some exam-style questions for this class on certain topics that I will give you. Your first task is to deeply understand and clearly describe the style of exam questions for this class and contrast it with homework problems based on examples I will give you.

Exam examples:

- Inside exam_5.pdf, problems 8 and 9 are exam problems.
- Inside exam_7.pdf, problems 7 and 8 are exam problems.
- Inside exam_8.pdf, problems 3 and 4 are exam problems. Problem 2 is NOT an exam problem.
- Inside exam_9.pdf, problem 2 is an exam problem. Problems 1 and 3 are NOT exam problems.

Homework examples:

- Inside hw1.pdf, all problems are homework problems.

Based on those examples, provide:

1. A summary of the types of questions that appear on exams (e.g. conceptual multiple choice, numerical short answer, etc.).
2. At least 10 distinctive traits of the exam problems, at least 5 of which must contrast with the traits of the homework problems.

Response:

Based on the provided documents, I have analyzed the exam style for this Deep Learning course. It is distinct from the homework style, favoring intuition, mechanism tracing, and conceptual "twists" over the lengthy, rigorous mathematical derivations found in the problem sets.

1. Summary of Exam Question Types

The exams rely heavily on testing whether a student can apply deep learning concepts to slightly modified or “toy” scenarios. The questions generally fall into these categories:

- **Mechanism Tracing (Numerical & Logical):** You are given specific vectors, matrices, or tokens and asked to manually compute the output of a layer (e.g., Attention or RNN). This tests if you understand the operation at a granular level.
 - **Design Critiques:** You are presented with a hypothetical architecture change (e.g., stacking RNNs vertically instead of horizontally) or a non-standard layer choice (e.g., Argmax attention) and asked to identify flaws, such as gradient flow issues.
 - **Visual & Geometric Reasoning:** Questions that require drawing output distributions, contours on a grid, or analyzing diagrams. This often applies to normalization layers or data geometry.
 - **Conceptual “Twists”:** The exam introduces a variation of a standard algorithm (e.g., “Ridge-Attention” or “Multiplicative Regularization”) and asks you to analyze its properties using standard principles.
 - **Complexity Analysis:** Multiple-choice questions regarding the Big-O computational cost of specific operations (e.g., inverting matrices vs. updating them).
 - **PyTorch/Implementation Logic:** Instead of writing code, students select the correct loss functions, layer types, or necessary optimization components (checkboxes/multiple choice).
-

2. Distinctive Traits of Exam Problems

Here are 10 distinctive traits of the exam problems. The first five explicitly contrast with the homework style.

Contrasting Traits (Exam vs. Homework) 1. “Toy” Numerical Execution vs. Theoretical Derivations

- **Exam:** Asks students to perform manual calculations on small, specific inputs (e.g., computing “Argmax attention” on specific 3D vectors or calculating reconstruction loss for a specific 2×2 matrix).
- **Homework:** Focuses on general abstract proofs, such as proving stability for *any* learning rate η or deriving convergence rates for general matrices.

2. Visual Intuition vs. Algebraic Rigor

- **Exam:** Requires drawing the results of operations. For example, drawing the 1-SD and 2-SD contours of a Gaussian distribution after Batch Normalization or correcting a Machine Translation diagram.

- **Homework:** Relies on algebraic manipulation to prove statements, such as showing the equivalence of two different optimization formulations.

3. Complexity (Big-O) vs. Convergence Analysis

- **Exam:** Tests the computational efficiency of an architecture. Questions ask for the Big-O cost of a “Ridge self-attention layer” or a rank-1 matrix update.
- **Homework:** Tests the *mathematical* convergence speed (e.g., “how many iterations t would we need?” or deriving the geometric convergence rate).

4. Implementation *Decisions* vs. Implementation *Coding*

- **Exam:** Tests knowledge of frameworks (PyTorch) through high-level decision making. For example, selecting the correct loss function (`nn.CrossEntropyLoss` vs `nn.MSELoss`) or necessary components (Weight Decay, Dropout) via checkboxes.
- **Homework:** Involves filling in lines of pseudocode for complex optimizers (Adam) or working through Jupyter notebooks to implement algorithms from scratch.

5. Architecture “Debugging” vs. Standard Derivation

- **Exam:** Presents a flaw or a change in a standard model and asks “What is the problem?” For instance, explaining why using `argmax` in attention breaks gradient flow, or why vertically stacking RNN inputs is problematic.
- **Homework:** asks students to derive the properties of a standard, working algorithm (e.g., deriving the update rules for standard Gradient Descent with Momentum).

Additional Exam Traits 6. The “Fusion” Problem The exams often fuse two distinct course concepts into a single novel problem. For example, “**Ridge-Attention**” combines Ridge Regression with the Attention mechanism, or “**Self-Supervised Linear Purification**” combines Autoencoders with SVD spectral filtering.

7. **Token-Level Data Flow** There is a specific focus on the exact inputs and outputs at specific timesteps. Questions ask for the exact list of 5 tokens passed into a decoder during training (checking for teacher forcing knowledge) versus evaluation.

8. **SVD as a Core Intuition Tool** While homeworks use SVD for proofs, exams use SVD for geometric intuition and filtering. Questions ask about preserving feature directions based on singular values (e.g., “preserve 80% of feature directions... while attenuating components”).

9. **Hyperparameter Reasoning** Exams ask for qualitative reasoning regarding hyperparameters without requiring a proof. For example, asking which

I don’t know if I like 3, 4, and 5 here as much; maybe I was too harsh to make it do at least 5 contrasts.

However, I do like the first two, and I think they already provide a good contrast between homework and exam.

Number 7 seems quite specific, but the other “traits” summarized seem reasonable to me.

regularization parameter λ causes a specific weight matrix to have higher loss , or determining the scaling factor s to normalize a dot product.

10. “Fill-in-the-Blank” Conceptual Math Rather than deriving an equation from scratch, exams often provide the structure of the equation and ask students to identify the missing terms (e.g., identifying that a noise term corresponds to μ or σ^2 in a regularized loss function).

Prompt:

hw01_solution - PDF

Now that you’re familiarized with the style of the exam problems, your second task is to understand the boundaries of the “scope” of the exam problems I want you to write. I will give you a problem set (and its solution), and I want the scope of the exam problems you write later to be about the topics in the homework.

Your current task is to read the homework solutions, and identify:

1. What big conceptual topics this homework covers (e.g. stochastic gradient descent)
2. What small conceptual details this homework covers (e.g. for analysis, it can be helpful to re-parameterizing the weight vector by subtracting the optimal weight vector from it, and then left-multiplying by the right singular matrix transposed).

Response:

Based on the solutions provided in `hw01_solution.pdf`, here is the breakdown of the scope for the exam problems. The homework focuses heavily on the theoretical underpinnings of optimization dynamics (GD, SGD, Momentum) and the probabilistic interpretations of regularization.

1. Big Conceptual Topics

- **Gradient Descent (GD) Dynamics & Stability:** Understanding the conditions under which Gradient Descent converges or diverges based on learning rates and the curvature (eigenvalues) of the loss landscape. This includes analyzing convergence rates for both scalar and vector/parallel problems.
- **Stochastic Gradient Descent (SGD) in Interpolation Regimes:** Analyzing SGD behavior specifically when the model is over-parameterized (wide matrices) and can achieve zero training loss (interpolation). This covers why constant step-size SGD can converge exponentially in these specific settings, unlike traditional convex optimization views.

The “big conceptual topics” summary seems to follow the problems (like in terms of ordering), which I think is reasonable, and they are good summaries.

- **Momentum & Acceleration:** How adding a momentum term (β) changes the recurrence relation of optimization. This includes understanding how momentum allows for larger learning rates and faster convergence, particularly in ill-conditioned problems (large ratio between max and min singular values).
- **Regularization & Robustness:** The mathematical equivalence between different forms of regularization. Specifically:
 1. **Noise Injection:** Adding noise to input data during training is equivalent to L2 (Tikhonov) regularization.
 2. **Weight Decay vs. L2:** The equivalence between adding a penalty to the loss function vs. decaying weights during the update step.
- **Bayesian Interpretations of Linear Models:** Viewing Ridge Regression through the lens of Maximum A Posteriori (MAP) estimation. This includes the duality between the Primal form (standard Ridge) and the Dual form (Kernel Ridge) derived via Gaussian conditioning.

2. Small Conceptual Details

These are the specific “tricks,” algebraic manipulations, and granular insights required to solve the problems. These are prime candidates for the “mechanism tracing” or “fill-in-the-blank” style exam questions.

- **Coordinate Shifts for Analysis:** To analyze convergence, one should re-parameterize the weight vector by subtracting the optimal weight ($w' = w - w^*$) to set the fixed point to zero.
 - *Exam Application:* A question might provide a non-zero centered problem and ask for the recurrence relation of the error vector.
- **SVD for Diagonalization:** Using Singular Value Decomposition (SVD) ($X = U\Sigma V^T$) to rotate the problem into a basis where the variables are decoupled (diagonalized). This allows treating a matrix problem as n independent scalar problems.
 - *Exam Application:* Asking for the specific matrix V required to diagonalize a given toy loss landscape.
- **The “Effective” Square System:** In over-parameterized settings (wide X), the analysis can be reduced to a square invertible system ($\tilde{X}$) because the updates in the null-space directions remain zero if initialized at zero.
- **Lyapunov Functions for SGD Proofs:** Using a specific quadratic Lyapunov function $L(\tilde{w}) = \tilde{w}^T \tilde{X}^T \tilde{X} \tilde{w}$ to prove convergence. The analysis decomposes the update into a linear contraction term A and a quadratic noise term B .
- **Momentum State-Space & Eigenvalues:** Writing Momentum updates as a matrix recurrence relation $[a_{t+1} x_{t+1}] = R_i [a_t x_t]$.
 - *Critical Insight:* The eigenvalues of this matrix R determine stability. Momentum is unique because it can create **complex conjugate eigenvalues**, where the convergence rate $\sqrt{1-\beta}$ is independent of

I like the small conceptual details it identified.

the learning rate η and data curvature σ .

- **Regularization Equivalence Constants:**
 - For **Weight Decay**, the decay factor γ is related to the L2 coefficient λ and learning rate η by $\gamma = \eta\lambda$.
 - For **Input Noise**, the equivalent L2 penalty λ is equal to the noise variance σ^2 multiplied by the dataset size m (or just σ^2 depending on scaling).
- **Gaussian Conditioning Formula:** Deriving the dual solution requires the property that for zero-mean joint Gaussians, $E[W | Y = y] = \Sigma_{WY} \Sigma_{YY}^{-1} y$. Computing the cross-covariance Σ_{WY} and auto-covariance Σ_{YY} is the key mechanical step.

Prompt:

Now that you have learned both the style and the content for the exam problems you should write, it's finally time to do your job to write the exam problems as an exam-writing TA.

Before writing the exam, note that you are NOT required to cover all types of exam problems, you are NOT required to cover all conceptual details present in the homework example, and you are NOT required to distribute the conceptual details to the different problems in the same way as did the homework.

Now, please think of some exam-style problems (recall your initial analysis of what exam-style means), with the scope (topics and conceptual details) that we just decided. You have creative freedom for how you write your problems, but they should be in the same "spirit" as the exam problems. Outside of having a distinctive style, this also means that the problems should:

1. Be finishable relatively quickly as long as the student has built intuition from the homeworks (and for more difficult problems, include a hint).
2. Should not expect students to memorize specific results from homeworks, and should fully clarify settings even if a setting is similar to a homework problem, although the exam can expect students to remember conceptual details of techniques.

Now, give me latex for the exam problems. For now, please omit the solutions (that will be later).

Response:

Here are the exam problems based on the scope of Homework 1. I have designed these to test the specific conceptual details you identified (Momentum eigenvalues, Noise-Regularization equivalence, SGD initialization properties, and MAP derivation) using the "toy problem" and "design critique" style typical of the exams.

As I mentioned before, difficulty after this step can vary a lot depending on the exact prompt. I reserve more specific analysis to annotations of the exam it outputted.

Exam Problems - Nov 28, 3:04 PM

Summary of Problems:

- **Problem 1 (Momentum):** A numerical application of the stability analysis. Instead of a proof, students must calculate the system matrix for a 1D case and identify the “critical damping” point numerically.
- **Problem 2 (Regularization):** A “Debugging” style question where students must map noise injection (Input vs. Label) to its equivalent analytical loss function.
- **Problem 3 (SGD Analysis):** A geometric reasoning problem. It tests the concept that SGD updates live in the row space of X , requiring a manual calculation of convergence for a non-zero initialization.
- **Problem 4 (Bayesian):** A “Fill-in-the-blank” derivation. It twists the standard Ridge derivation by changing the prior mean and noise covariance, checking if students understand which part of the probabilistic model affects which part of the loss function.

Prompt:

Now that the problems are finished, please write the solutions for the exam. The solutions file should include:

1. The original problems.
2. The solutions, preferably in a different color.
3. Rubrics (e.g. percentage of credit for doing a certain thing), can be short, preferably in a different color.

Please give the solutions also in latex.

Response:

The following LaTeX file contains the exam problems, solutions, and grading rubrics.

Key features of the solutions:

- **Visual Distinction:** Solutions and rubrics are colored in **blue** to distinguish them from the problem text.
- **Step-by-Step Logic:** The derivations (especially for Problem 1 and 3) are written out explicitly, mirroring the expectation for “showing work” on the exam.
- **Rubrics:** Short, actionable scoring guides are provided for each sub-problem. > **Exam Solutions** - Nov 28, 3:13 PM

I again reserve more specific analysis to annotations of the exam it outputted.

Prompt:

Response:

It seems there was a compilation error in the LaTeX code I generated, specifically regarding a special character in the header. I will resolve this error now and regenerate the correct file for you; this should just take a moment.